

BadCourt — Phase 0: Foundation

Phase: 0 of 10 — Foundation

Shape: 21 steps across 5 stages, one commit per step

Exit criteria: `dotnet test` green; Aspire dashboard shows a healthy app

Date: 2 September 2026

1. Scope

Phase 0 delivers the skeleton every later phase copies: the solution and its build configuration, the SharedKernel primitives, the CQRS dispatcher and its decorator pipeline, the API host, the database roles that make the read/write split real, the Aspire orchestration, and CI.

It deliberately contains **no domain logic**. Nothing here knows what a court or a booking is. The first module (Identity) arrives in Phase 1 and becomes the template repeated six times, so the foundation it sits on must be correct first.

Exit criteria (from the master plan): `dotnet test` green; Aspire dashboard shows a healthy app.

2. Ground rules

Each step is one concern, ends in a command that either passes or fails, and lands as its own commit, so the diff can be reviewed in isolation.

- **One concern per step.** If a step needs the word "and" to describe it, it should probably be two steps.
- **One commit per step**, so `git show` is a reviewable unit.
- **Every step ends in a verification command.** A step with nothing to run is a step whose failure surfaces three steps later, attached to the wrong cause.
- **Tests arrive with the code they test**, not in a batch at the end. The unit-test project is created at step 3, not step 18.
- **Warnings are errors** repo-wide from step 2 onward, so nothing accumulates.

Starting state

`git ls-files` lists five files: `.gitignore`, `BadCourt.slnx`, `docs/build-pdf.py`, `docs/implementation-plan.md`, `docs/implementation-plan.pdf`. A previous Phase 0 attempt sits in `stash@{0}` as a reference. `src/`, `tests/` and `TestResults/` still exist on disk holding only gitignored `bin / obj` output from that attempt — stale DLLs on the probing path, removed by step 0.

3. Stage A — Toolchain

No C# yet. This stage exists so that when step 3 runs the first test, a failure is unambiguously about the test rather than the runner.

#	Deliverable	Verification
0	Delete stale <code>src/</code> , <code>tests/</code> , <code>TestResults/</code> build output	<code>find src tests -type f</code> returns nothing
1	<code>global.json</code> , <code>.editorconfig</code> , <code>.gitattributes</code>	<code>dotnet --version</code> prints the pinned SDK
2	<code>Directory.Build.props</code> , <code>Directory.Packages.props</code> , empty <code>BadCourt.slnx</code>	<code>dotnet build BadCourt.slnx</code> succeeds with zero projects

Step 1 notes. `global.json` pins SDK `10.0.400` with `rollForward: latestFeature`, and carries the `Microsoft.Testing.Platform` opt-in:

```
{
  "sdk": { "version": "10.0.400", "rollForward": "latestFeature" },
  "test": { "runner": "Microsoft.Testing.Platform" }
}
```

That `test` block is the only location the .NET 10 SDK reads it from — `dotnet.config` and the `TestingPlatformDotnetTestSupport` / `UseMicrosoftTestingPlatformRunner` MSBuild properties have no effect.

`.gitattributes` sets a CRLF working tree with LF forced for `db/init/*.sql`, `docker-compose.yml`, `.github/workflows/*.yml` and `*.sh`. This is not cosmetic: a carriage return inside a psql meta-command line becomes part of the variable name, and one in a workflow file leaks into the Linux runner's shell.

`.editorconfig` field-naming rules must declare the `const` and `static readonly` cases **before** the catch-all instance-field rule, because the first matching rule wins — otherwise `dotnet format` demands a leading underscore on constants.

Step 2 notes. `Directory.Packages.props` deliberately omits `Microsoft.NET.Test.Sdk` and `xunit.runner.visualstudio`; `xunit.v3` supplies the `Microsoft.Testing.Platform` entry point, and the VSTest bridge is no longer supported on the .NET 10 SDK.

4. Stage B — SharedKernel, test-first

#	Deliverable	Verification
3	<code>Result</code> , <code>Result<T></code> , <code>Error</code> — plus the <code>BadCourt.UnitTests</code> project	<code>dotnet test</code> → ~6 green
4	<code>Entity</code> , <code>AggregateRoot</code> , <code>IDomainEvent</code> , <code>PagedList<T></code>	Identity-based equality; domain-event collection
5	<code>ICommand</code> , <code>ICommand<T></code> , <code>IQuery<T></code> , handler interfaces, <code>ISender</code>	Compiles; no implementation yet
6	<code>Dispatcher</code> + <code>AddMessaging(assembly)</code> Scrutor registration	A command and a query each dispatch to their handler
7	Logging decorator	Asserted through a fake logger
8	Validation decorator (<code>FluentValidation</code>)	Invalid input yields a failed <code>Result</code> ; the handler never runs
9	Transaction decorator + <code>ITransactionScopeFactory</code> seam	Commit on success, rollback on handler failure
10	Pipeline order test	Exact sequence assertion

Step 3 is the first real test of the toolchain. `Result` is chosen for it because it is pure, dependency-free, and the type every handler in the system returns.

Steps 6–10 test against the **real** container produced by `AddMessaging`, with only the transaction seam faked. A hand-assembled pipeline in a test proves that a hand-assembled pipeline works; it says nothing about the one the application runs.

Step 10 pins the decorator order as **Logging** → **Validation** → **Transaction** → **Handler**.

5. Stage C — Composition

#	Deliverable	Verification
11	<code>BadCourt.Integration</code> — <code>IIntegrationEvent</code> + in-process bus	One publish reaches two handlers
12	<code>BadCourt.ServiceDefaults</code> — <code>OpenTelemetry</code> , health checks, resilience	Builds
13	<code>BadCourt.Api</code> — host, <code>PingController</code> , <code>Result</code> → <code>ProblemDetails</code> mapping, <code>Scalar</code> , <code>launchSettings.json</code>	<code>dotnet run</code> , then <code>GET /api/ping</code>
14	<code>BadCourt.ArchitectureTests</code> — layering and controller-placement rules	5 green

Step 13 notes. Controllers rather than Minimal APIs, contributed per module via an MVC `ApplicationPart`, so each module's `Presentation` project stays self-contained.

Step 14 notes. The project needs

```
<Using Include="NetArchTest.Rules.TestResult" Alias="TestResult" />
```

, because `NetArchTest.Rules.TestResult` and `Xunit.TestResult` are both pulled in by global usings and collide (CS0104). The assembly list is structured so each later phase appends its module assemblies to one array.

6. Stage D — Database

#	Deliverable	Verification
15	<code>db/init/01-roles-and-database.sql</code> , <code>db/init/02-schemas-and-grants.sql</code>	Role listing, and <code>SHOW default_transaction_read_only</code> as each role
16	<code>docker-compose.yml</code> — Postgres + PostGIS, Mailpit	Container reports healthy
17	<code>IReadConnectionFactory</code> + <code>Database</code> options binding	Options-validation unit test
18	<code>BadCourt.IntegrationTests</code> — Testcontainers, role split, <code>/health</code> , <code>/api/ping</code>	5 green with Docker; 5 skipped without

Step 15 notes. Passwords reach the scripts as environment variables read through psql meta-commands and interpolated with psql's own quoting syntax, so psql performs the escaping and a password containing a quote cannot inject SQL. The scripts create both roles, the `badcourt` database owned by `badcourt_app` , the PostGIS and `btrees_gist` extensions, six module schemas, and — the point of the exercise —

`ALTER ROLE badcourt_read SET default_transaction_read_only = on` , together with `ALTER DEFAULT PRIVILEGES` so future tables are readable without further grants.

Step 17 notes. `IReadConnectionFactory` returns `DbConnection` , not `NpgsqlConnection` , keeping Npgsql out of SharedKernel. Dapper extends `IDbConnection` , so nothing is lost. An architecture test enforces it.

Step 18 notes. The fixture must construct the container **inside** its `try / catch` , not in a field initializer: building a Testcontainers container probes the Docker endpoint, so an absent daemon throws during construction and turns five clean skips into five misleading failures. The tests mount the same `db/init` scripts the AppHost uses, located by walking up to the directory containing `BadCourt.slnx` , so the tested schema and the running schema cannot drift.

The centrepiece is the test that justifies the entire read/write split: `badcourt_read` attempting `CREATE TABLE` must fail with `SqlState 25006` .

7. Stage E — Orchestration and CI

#	Deliverable	Verification
19	<code>BadCourt.AppHost</code> — Postgres + PostGIS, Mailpit, Cloudinary parameter, both connection strings	<code>aspire run</code> → dashboard all green
20	<code>.github/workflows/ci.yml</code> , plus the same four commands run locally	restore → format → build → test all exit 0

Step 19 notes. The only step that requires Docker Desktop running. It is also the one genuine unknown in Phase 0: whether `WithInitFiles` coexists with the init script's `CREATE DATABASE badcourt` when Aspire's `AddDatabase("badcourt")` expects to create the same database.

Both role passwords are derived from the Postgres resource's own password parameter, so no credential is written to a file that git can see.

8. Deviations from the master plan

Three, all recorded back into `implementation-plan.md`:

- 1. **Decorator order.** The master plan places validation innermost, which opens a database transaction before a malformed request is rejected. Validation touches no database, so it belongs outside the transaction. Step 10 pins the corrected order.
- 2. **`IReadConnectionFactory` returns `DbConnection`.** Returning `NpgsqlConnection` would drag Npgsql into SharedKernel and break the layering rule that SharedKernel depends on nothing.
- 3. **Cloudinary replaces MinIO.** The Aspire MinIO hosting integration is obsolete, and the predecessor already used Cloudinary. Being a hosted service it needs no container — only a secret, in Cloudinary's own URL format, which collapses the predecessor's three settings into one.

9. Known traps

Symptom	Cause
<code>dotnet test</code> reports "Zero tests ran", exit code 5, ~200 ms	<code>--noLogo</code> is forwarded to the test application, which rejects it. Do not pass it under <code>Microsoft.Testing.Platform</code> .
"Testing with VSTest target is no longer supported"	The MTP runner opt-in belongs in <code>global.json</code> , nowhere else.
Five integration tests fail rather than skip when Docker is down	The container was built in a field initializer, and building probes the Docker endpoint.
CS0104: 'TestResult' is an ambiguous reference	<code>NetArchTest.Rules.TestResult</code> versus <code>Xunit.TestResult</code> under global usings.
<code>dotnet format</code> demands <code>_</code> on constants	<code>.editorconfig</code> catch-all field rule declared before the <code>const</code> / <code>static readonly</code> rules.
psql init script fails on an unknown variable	CRLF line endings in <code>db/init/*.sql</code> .

10. Checklist

Stage A	☐ 0 clean	☐ 1 toolchain	☐ 2 build config
Stage B	☐ 3 Result	☐ 4 Entity	☐ 5 abstractions
	☐ 6 dispatcher	☐ 7 logging	☐ 8 validation
	☐ 9 transaction	☐ 10 order	
Stage C	☐ 11 Integration	☐ 12 ServiceDefaults	☐ 13 Api
	☐ 14 arch tests		
Stage D	☐ 15 db init	☐ 16 compose	☐ 17 read connection
	☐ 18 integration tests		
Stage E	☐ 19 AppHost	☐ 20 CI	

11. Prerequisite carried into this phase

Cloudinary credentials must be **newly issued**, with the predecessor's pair revoked on Cloudinary's side rather than merely left unused — the old key and secret are in `se121.badcourt`'s git history, and Phase 0 wires BadCourt to the same service. Once issued:

```
dotnet user-secrets --project src/api/BadCourt.AppHost set "Parameters:cloudinary-url"
"cloudinary://key:secret@cloud"
```

Nothing reads the value until Phase 2; a placeholder default keeps `aspire run` working until then.

End of document.